



TrixiCUDA.jl: CUDA Support for Solving Hyperbolic PDEs on GPU

Huiyu Xie (@huiyuxie)
JuliaCon 2025

** We are using Trixi.jl v0.11.17 and
TrixiCUDA.jl v0.1.0-rc.3 for this talk.*

Two Preliminary Questions

What is hyperbolic PDE?

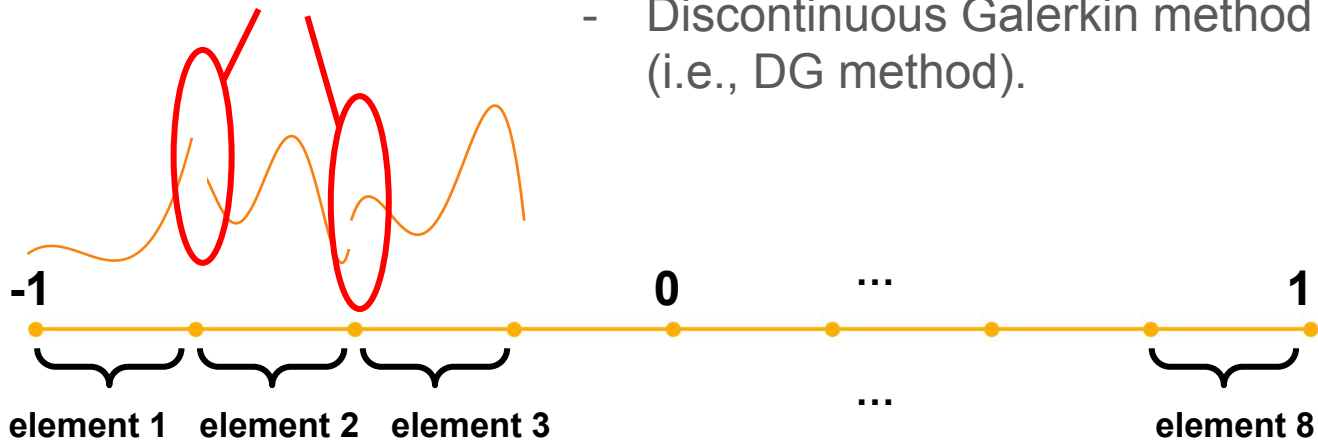
- Wave-like behaviors (e.g., wave equation).

Apply numerical flux.

Why hyperbolic PDEs + GPU?

- Discontinuous Galerkin method (i.e., DG method).

DG allows discontinuity.



Simple 1D example with DG



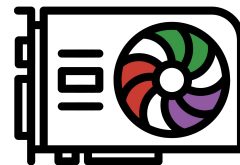
Trixi-Framework



Trixi-GPU

Good combination: Julia and CUDA.

Core acceleration: Semidiscretization.



JuliaGPU

What is semidiscretization?

- Discretization in space (i.e., spatial discretization).

Why semidiscretization?

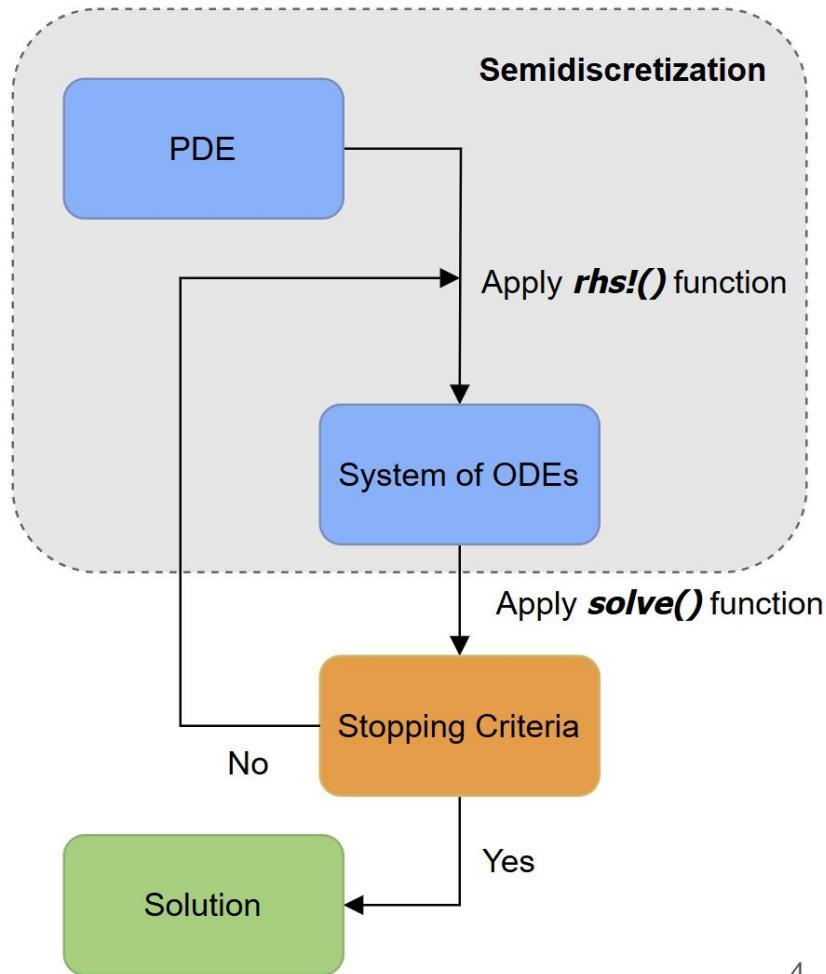
- Discontinuous Galerkin (massive, matrix-free parallelism).

juliacon Workflow Skeleton

rhs_gpu!(): Semidiscretization on GPU, encapsulating all GPU kernels.

- **cuda_volume_integral!()** computes volume integral on GPU.
- **cuda_interface_flux!()** computes interface flux on GPU.
- **cuda_boundary_flux!()** computes boundary flux on GPU.
- Other GPU kernels...

* See tutorial: [Introduction to DG methods](#) for volume integral, interface flux, boundary flux, etc.



Public API design principles: Similar and intuitive

```
# Set up equation
equations = LinearScalarAdvectionEquation1D(...)

# Create DGSEM solver
solver = DGSEM(...)

# Set up tree mesh
mesh = TreeMesh(...)

# Create semidiscretization
semi = SemidiscretizationHyperbolic(...)

# Create ODE
ode = semidiscretize(...)

# Set up callbacks
callbacks = CallbackSet(...)

# Solve ODE
sol = solve(...)
```

Trixi.jl APIs

```
# Set up equation
equations = LinearScalarAdvectionEquation1D(...)

# Create DGSEM solver (GPU-enabled)
solver = DGSEMGPU(...)

# Set up tree mesh
mesh = TreeMesh(...)

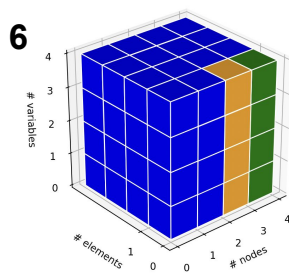
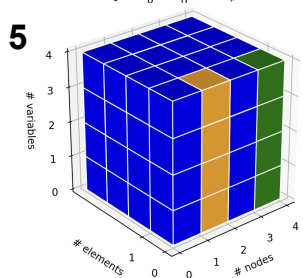
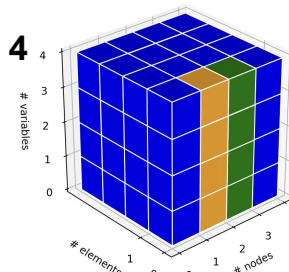
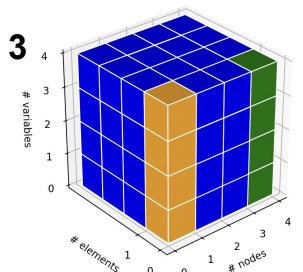
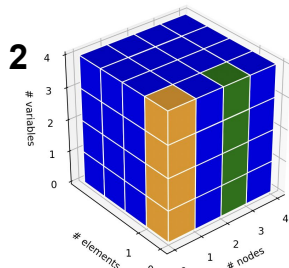
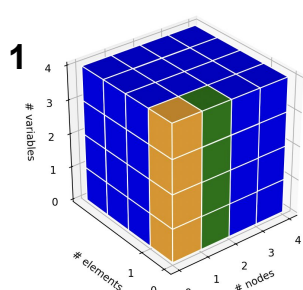
# Create semidiscretization (GPU-enabled)
semi = SemidiscretizationHyperbolicGPU(...)

# Create ODE (GPU-enabled)
ode = semidiscretizeGPU(...)

# Set up callbacks
callbacks = CallbackSet(...)

# Solve ODE
sol = solve(...)
```

TrixiCUDA.jl APIs



Data: Current state u ($4 \times 4 \times 4$ data grid)

Example: Volume integral with flux differencing for 1D problems

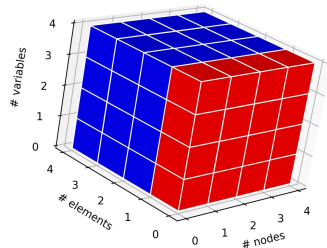
Pseudocode **Dependent loops!**
for element in 1: #(elements):

for idx1 in 1: #(nodes):
for idx2 in (idx1+1): #(nodes):

compute volume flux between nodes
idx1 and idx2

accumulate volume flux into node idx1

end
end
end



Data: Single layer

* See tutorial: [DGSEM with flux differencing](#) for flux differencing.

Pseudocode **Dependent loops!**

```
for element in 1: #(elements):
```

```
  for idx1 in 1: #(nodes):
```

```
    for idx2 in (idx1+1): #(nodes):
```

```
      // compute flux...
```

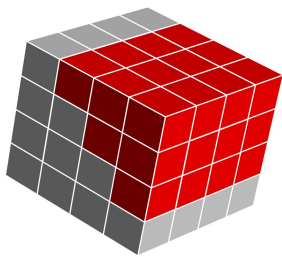
```
    end
```

```
  end
```

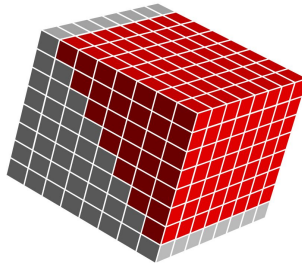
```
end
```

Naive approach: Launch thread grid
 $\#(\text{elements}) \times \#(\text{nodes}) \times \#(\text{nodes})$.

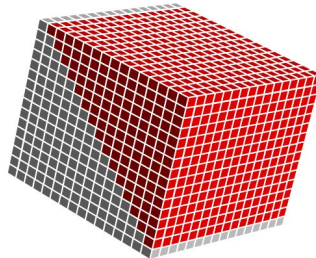
SIMD disables threads corresponding to
 1:idx1 in the innermost loop, and enables
 the remaining threads.



4×4×4 threads



8×8×8 threads



16×16×16 threads

...

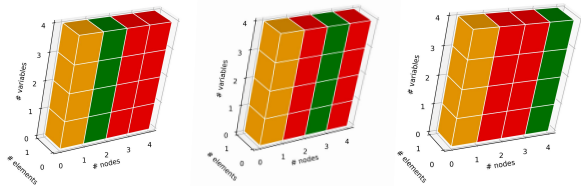
n×n×n threads

$\# \text{grey} / (\# \text{grey} + \# \text{red}) = (n-1)/2n$ **Nearly half of threads are doing nothing!**

Optimizations:

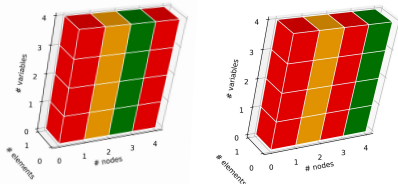
- Tiling with shared memory
- Thread coarsening

T1



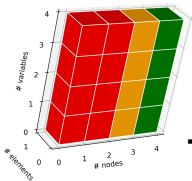
+ data loading time ≈ 4 time units

T2



+ data loading time ≈ 3 time units

T3

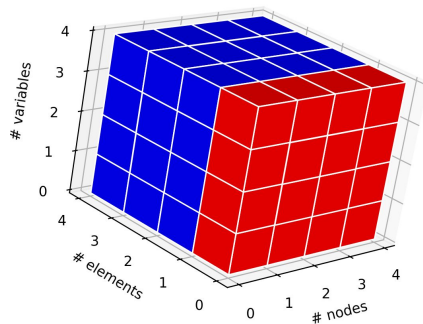


+ data loading time ≈ 2 time units

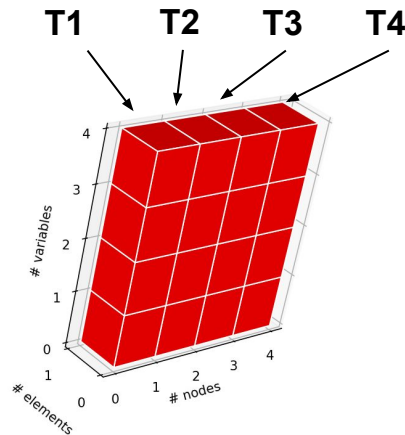
T4 ≈ 1 time unit

Step 1: Tile the data grid by element layers and load each layer into shared memory.

Step 2: In each tile, map each dependent loop to a single thread (i.e., thread coarsening).



Red: Single shared memory tile

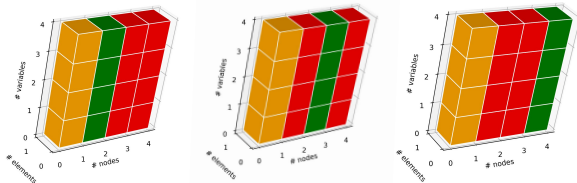


Thread mapping per block

Nearly half the time is idle!

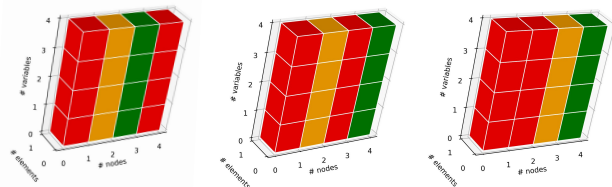
Improvement #1: Consolidate each complementary pair of threads into a single thread.

$$*T1 = T1 + T4$$



+ data loading time ≈ 5 time units

$$*T2 = T2 + T3$$

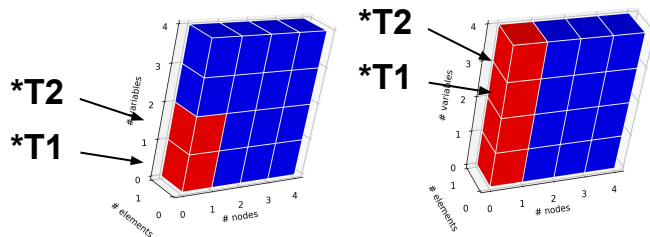


+ data loading time ≈ 5 time units

This frees up computing resources for other jobs!

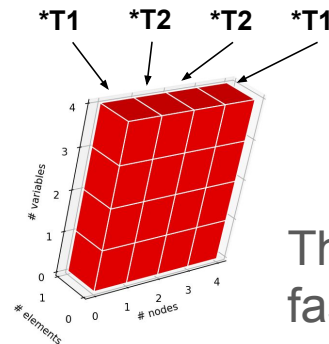
Improvement #2: Coalesce data transfers from global to shared memory.

Load data



** Data loading is typically coalesced across 32 threads in practice.*

Compute data



** Transpose all the threads when computing the data.*

This makes data transfer faster!

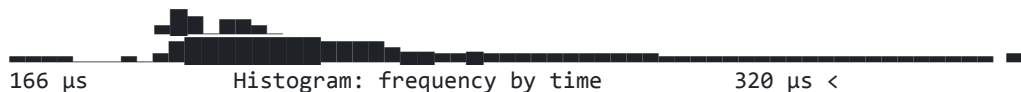
Apply optimizations similarly to 1D, 2D, and 3D problems...

Benchmarks: 3D Euler equations with entropy conservative flux

BenchmarkTools.Trial: 10000 samples with 1 evaluation.

Range (min ... max):	166.000 μ s ... 21.309 ms	GC (min ... max):	0.00% ... 18.31%
Time (median):	207.700 μ s	GC (median):	0.00%
Time (mean $\pm$ σ):	223.133 μ s $\pm$ 412.949 μ s	GC (mean $\pm$ σ):	0.68% $\pm$ 0.36%

Before the optimizations

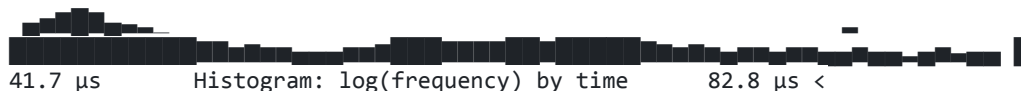


Memory estimate: 8.58 KiB, allocs estimate: 124.

BenchmarkTools.Trial: 10000 samples with 1 evaluation.

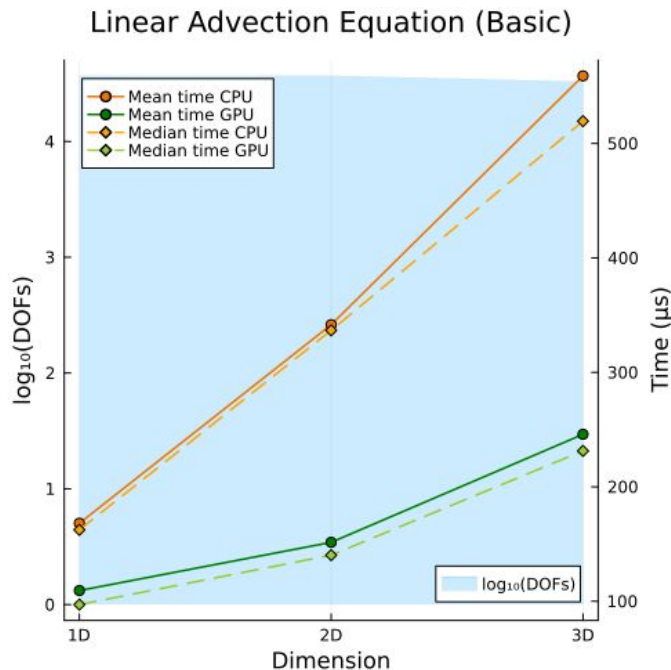
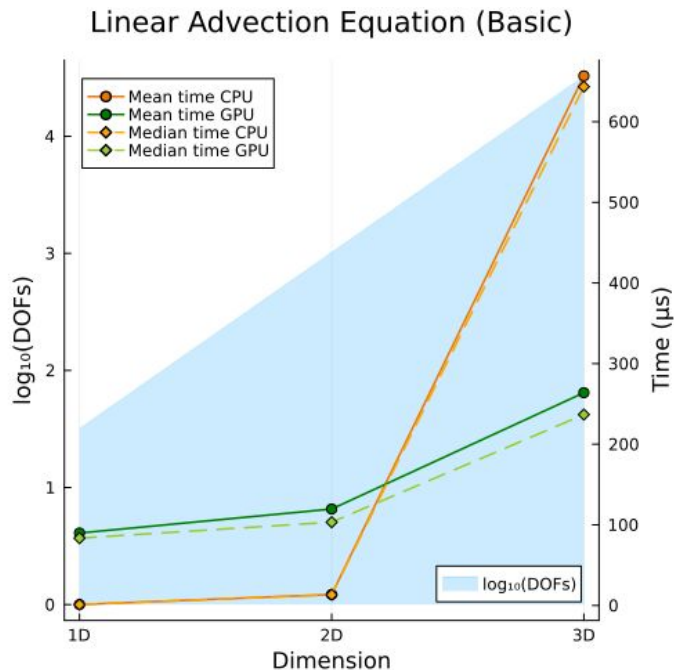
Range (min ... max):	41.700 μ s ... 260.800 μ s	GC (min ... max):	0.00% ... 0.00%
Time (median):	44.900 μ s	GC (median):	0.00%
Time (mean $\pm$ σ):	47.063 μ s $\pm$ 9.970 μ s	GC (mean $\pm$ σ):	0.00% $\pm$ 0.00%

After the optimizations



Memory estimate: 2.52 KiB, allocs estimate: 41.

** Actual implementations vary by data size. See [PR #105](#) for details and more benchmarks.*



GPU advantage grows with problem complexity.

* We are using *Trixi.jl* v0.11.17 and *TrixiCUDA.jl* v0.1.0-rc.3 for the benchmark.

Takeaways about TrixiCUDA.jl:

- Julia and CUDA: Combination of the best of both worlds.
- Package: GPU accelerator package, single precision support, intuitive and similar APIs.
- Acceleration optimization: Semidiscretizations, Discontinuous Galerkin methods, speedups on high-dimension or high-resolution problems.

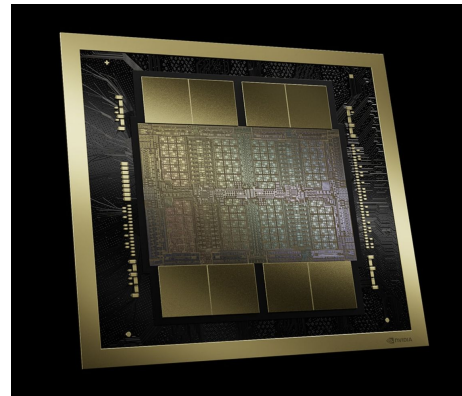


Key challenge:

General optimization across diverse problems and their inputs (related to autotuning, but not exactly), and GPU architectures.

Future directions:

1. Extend meshes, callbacks, time-stepping, and more to the GPU for full acceleration.
2. Implement mixed-precision algorithms (semidiscretization and more).



Blackwell architecture



Highlights and Gaps in CUDA.jl

CUDA ecosystem

- Package ecosystem: cuBLAS, cuFFT, cuSOLVER, ...
 - Choose standard math functions / intrinsic functions, [reference](#).
 - Support more types (e.g., `float4`, `double2`, etc.), [reference](#).
 - Not a big problem!
- Tool ecosystem: CUDA-GDB, Nsight Compute, Nsight System, ...
 - Map SASS/PTX correctly to the original kernel code, rather than to lower-level function calls.
 - [Godbolt example](#).
 - Provide full CUDA-GDB support in CUDA.jl.



Acknowledgment

Project Advisors

- Prof. Hendrik Ranocha (Johannes Gutenberg University Mainz, Germany)
- Prof. Jesse Chan (Oden Institute - University of Texas at Austin, U.S.)
- Prof. Michael Schlottke-Lakemper (University of Augsburg, Germany)

Upstream Developers

- Tim Besard (Lead Developer, JuliaGPU)
- Christopher Rackauckas (Lead Developer, SciML)

Julia Community

Larsson, S., Thomee, V. (2008). *Partial Differential Equations with Numerical Methods*. Germany: Springer Berlin Heidelberg.

Hesthaven, J. S., Warburton, T. (2008). *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*. United Kingdom: Springer.

Hwu, W. W., Kirk, D. B., El Hajj, I. (2022). *Programming Massively Parallel Processors: A Hands-on Approach*. Netherlands: Morgan Kaufmann.

Trixi.jl Documentation, <https://trixi-framework.github.io/TrixiDocumentation/stable/>, accessed June 2025.

CUDA.jl Documentation, <https://cuda.juliagpu.org/stable/>, accessed June 2025.

SciML Documentation, <https://docs.sciml.ai/Overview/stable/>, accessed June 2025.



Thank You!

Seeking Ph.D. Position for Fall 2026
Contact at huiyuxie.sde@gmail.com



** Scan it or go to*
<https://trixi-gpu.github.io/assets/files/juliacon25.pdf>
to access the slides.